

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	RPC Vs. gRPC				
NÚMERO DE PRÁCTICA:	05	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	11/05/2026	FECHA FIN:	15/05/2026	DURACIÓN:	2 horas
RECURSOS A UTILIZAR: Entorno de desarrollo: Visual Studio Code o IntelliJ IDEA, Java Development Kit, Maven, Protocol Buffers gRPC, Postman, Docker (opcional)					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender las diferencias conceptuales entre RPC tradicional y gRPC. - Implementar servicios distribuidos usando RPC y gRPC. - Evaluar rendimiento y escalabilidad entre ambos enfoques. - Analizar ventajas y limitaciones arquitectónicas de cada modelo. - Aplicar buenas prácticas de serialización y comunicación remota.	
TEMAS: RPC tradicional y gRPC.	
COMPETENCIA	C.e. Identifica de forma reflexiva y responsable, necesidades a ser resueltas usando tecnologías de información y/o desarrollo de software en los ámbitos local, nacional o internacional, utilizando técnicas, herramientas, metodologías, estándares y principios de la ingeniería

CONTENIDO DE LA GUÍA

1. RPC (Remote Procedure Call)

RPC permite que un programa invoque procedimientos alojados en otro nodo como si fueran locales.

Características:

- Transparencia de ubicación
- Comunicación cliente-servidor
- Uso frecuente de JSON/XML
- Mayor latencia en algunos escenarios

Ventajas:

- Fácil implementación
- Buena interoperabilidad

Desventajas:

- Menor eficiencia
- Serialización pesada

2. gRPC

gRPC es un framework moderno desarrollado por Google que utiliza HTTP/2 y Protocol Buffers.

Características:

- Comunicación binaria eficiente
- Multiplexación HTTP/2
- Streaming bidireccional
- Alto rendimiento

Ventajas:

- Baja latencia
- Excelente escalabilidad
- Contratos estrictos

Desventajas:

- Curva de aprendizaje mayor
- Menor legibilidad directa

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Implementación de Calculadora Distribuida usando gRPC

Paso 1: Archivo calculator.proto

```
syntax = "proto3";

service Calculator {
  rpc Sum (Request) returns (Response);
}

message Request {
  int32 a = 1;
  int32 b = 2;
}

message Response {
  int32 result = 1;
}
```

Paso 2: Servidor Java

```
public class CalculatorService extends CalculatorGrpc.CalculatorImplBase {

    @Override
    public void sum(Request req, StreamObserver<Response> responseObserver) {
        int result = req.getA() + req.getB();

        Response response = Response.newBuilder()
            .setResult(result)
            .build();

        responseObserver.onNext(response);
        responseObserver.onCompleted();
    }
}
```

Paso 3: Cliente Java

```
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("localhost",50051)
    .usePlaintext()
    .build();

CalculatorGrpc.CalculatorBlockingStub stub =
    CalculatorGrpc.newBlockingStub(channel);

Request request = Request.newBuilder()
```

```
.setA(8)
.setB(4)
.build();
```

```
Response response = stub.sum(request);
```

```
System.out.println("Resultado: " + response.getResult());
```

```
channel.shutdown();
```

Resultado esperado
Resultado: 12

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Ejercicio 1: Servicio RPC Tradicional

Implementar un servicio RPC cliente-servidor que reciba dos números y retorne:

- Multiplicación
- División
- Potencia

Solución guía (Java RMI)

```
public interface Calculator extends Remote {
    double multiply(double a, double b) throws RemoteException;
}
```

Implementación:

```
public double multiply(double a, double b){
    return a*b;
}
```

Deberá extender para división y potencia.

Ejercicio 2: Sistema de Conversión con gRPC

Implementar un servicio distribuido que convierta:

- Celsius → Fahrenheit
- Soles → Dólares
- Kilómetros → Millas

Archivo proto sugerido:

```
service Converter {
    rpc Convert (ConvertRequest)
    returns (ConvertResponse);
}
```

Servidor base:

```
@Override
public void convert(ConvertRequest req,
                    StreamObserver<ConvertResponse> responseObserver){

    double result = req.getValue()*1.8+32;

    responseObserver.onNext(
        ConvertResponse.newBuilder()
            .setResult(result)
            .build()
    );

    responseObserver.onCompleted();
}
```

El estudiante deberá:

- Añadir más conversiones
- Validar entradas
- Mostrar logs del servidor

ACTIVIDAD COMPARATIVA

Termine de realizar la tabla comparativa:

Métrica	RPC Tradicional	gRPC
Tiempo respuesta	___ ms	___ ms
Consumo memoria	___ MB	___ MB
Complejidad implementación	Alta/Media/Baja	Alta/Media/Baja
Escalabilidad	___	___

Analizar resultados.

Producto final esperado

El estudiante entregará:

- Código fuente cliente/servidor
- Archivo .proto
- Evidencias de ejecución
- Tabla comparativa de resultados
- Respuestas argumentadas del cuestionario

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1
Informe breve de conclusiones técnicas sobre RPC vs gRPC		

III. CUESTIONARIO

- ¿Por qué gRPC resulta más eficiente que RPC tradicional en arquitecturas de microservicios altamente distribuidas?
- ¿Qué limitaciones podría presentar gRPC en entornos donde la interoperabilidad humana (depuración manual o pruebas directas) es necesaria?
- Si diseñaras una plataforma bancaria distribuida, ¿qué factores arquitectónicos te harían elegir RPC tradicional o gRPC?

IV. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3]Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: AlfaomegaRa-Ma.
- Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wile

- <https://grpc.io/>
- <https://grpc.io/docs/>
- <https://grpc.io/docs/guides/>
- <https://protobuf.dev/overview/>
- <https://docs.oracle.com/javase/tutorial/rmi/>
- Baeldung gRPC Java Tutorial
- <https://github.com/grpc/>
- Thönes, J. (2015). *Microservices*. IEEE Software. DOI: 10.1109/MS.2015.11
- Dragoni et al. (2017). Microservices: Yesterday, Today, and Tomorrow. DOI: 10.1007/978-3-319-67425-4_12
- <https://codelabs.developers.google.com/grpc/getting-started-grpc-go?hl=es-419#4>
- <https://docs.cloud.google.com/api-gateway/docs/grpc-overview?hl=es-419>
- <https://adictosaltrabajo.com/2023/01/09/grpc-explicado-con-ejemplos-servidor-y-cliente/>

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Problemas/Ejercicios propuestos
/ Preguntas formuladas /
Resolución de casos*

INSTRUMENTOS:

Lista de cotejo, rúbrica.

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Deficiente (1)
Implementación técnica	Funciona completamente	Funciona parcialmente	Presenta errores menores	No funciona
Comprensión conceptual	Explica claramente diferencias	Explica parcialmente	Presenta vacíos conceptuales	No comprende
Resolución ejercicios	Completa ambos correctamente	Completa uno	Incompleto	No entrega
Análisis comparativo	Reflexión crítica profunda	Buena reflexión	Superficial	Ausente
Buenas prácticas de codificación	Código limpio y modular	Aceptable	Poco organizado	Desordenado

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1